

```

# =====
# DATABASE SETUP, SCHEMA DEFINITION, AND DATA LOADING
# =====
import sqlite3
import pandas as pd

# 1. Initialize the SQLite database connection
conn = sqlite3.connect('healthcare_capstone.db')
cursor = conn.cursor()

# 2. Enable foreign key constraints (required for SQLite)
cursor.execute("PRAGMA foreign_keys = ON;")

# 3. Define and execute the strict schema (DDL)
cursor.executescript("""
-- Drop tables if they exist to allow clean re-runs of this cell
DROP TABLE IF EXISTS billing;
DROP TABLE IF EXISTS visits;
DROP TABLE IF EXISTS patients;

-- Create Patients Table with constraints
CREATE TABLE patients (
    patient_id INTEGER PRIMARY KEY,
    age INTEGER CHECK (age >= 0 AND age <= 120),
    gender TEXT,
    city TEXT,
    insurance_provider TEXT,
    chronic_flag INTEGER CHECK (chronic_flag IN (0, 1)),
    registration_date TEXT
);

-- Create Visits Table with Foreign Key to Patients
CREATE TABLE visits (
    visit_id INTEGER PRIMARY KEY,
    patient_id INTEGER,
    visit_date TEXT,
    department TEXT,
    visit_type TEXT,
    length_of_stay_hours REAL,
    risk_score TEXT,
    doctor_id INTEGER,
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id) ON DELETE CASCADE
);

-- Create Billing Table with Foreign Key to Visits
CREATE TABLE billing (
    bill_id INTEGER PRIMARY KEY,
    visit_id INTEGER,
    billed_amount REAL,
    approved_amount REAL,
    claim_status TEXT,
    payment_days INTEGER,
    billing_date TEXT,
    FOREIGN KEY (visit_id) REFERENCES visits(visit_id) ON DELETE CASCADE
);

-- 4. Create the mandatory indexes as requested in the Capstone Intro
CREATE INDEX idx_visit_date ON visits(visit_date);
CREATE INDEX idx_department ON visits(department);
CREATE INDEX idx_insurance_provider ON patients(insurance_provider);
CREATE INDEX idx_claim_status ON billing(claim_status);
""")

# 5. Load data from the CSV files into the SQL tables
# Note: Ensure patients.csv, visits.csv, and billing.csv are uploaded to Colab files
pd.read_csv('patients.csv').to_sql('patients', conn, if_exists='append', index=False)
pd.read_csv('visits.csv').to_sql('visits', conn, if_exists='append', index=False)
pd.read_csv('billing.csv').to_sql('billing', conn, if_exists='append', index=False)

print("Phase 1 Setup Complete: Database built, schema enforced, and CSVs loaded.")

```

Phase 1 Setup Complete: Database built, schema enforced, and CSVs loaded.

```

# =====
# OPERATIONAL TASK 1 - Total Visits & Avg Length of Stay (LOS)
# =====
query_op1 = """
-- Calculate volume and average stay duration per department
SELECT
    department,

```

```
        COUNT(visit_id) AS total_visits,
        ROUND(AVG(length_of_stay_hours), 2) AS avg_los_hours
FROM visits
GROUP BY department
ORDER BY total_visits DESC;
"""
df_op1 = pd.read_sql(query_op1, conn)
display(df_op1)
```

	department	total_visits	avg_los_hours	
0	General	4228	19.43	
1	ER	4220	19.53	
2	Neurology	4165	19.72	
3	Orthopedics	4164	19.66	
4	Cardiology	4159	19.60	
5	ICU	4064	19.36	

Next steps: [Generate code with df_op1](#) [New interactive sheet](#)

Business Insight:

This baseline volume metric reveals patient flow distribution. Departments showing both high visit volumes and prolonged average lengths of stay represent our primary operational bottlenecks. Hospital administration must evaluate workflow efficiencies in these specific units to improve bed turnover rates.

```
# =====
# OPERATIONAL TASK 2 - High-Risk Patient Volume by Department
# =====
query_op2 = """
-- Identify which departments handle the most 'High' risk patients
SELECT
    department,
    COUNT(DISTINCT patient_id) AS high_risk_patient_count
FROM visits
WHERE risk_score = 'High'
GROUP BY department
ORDER BY high_risk_patient_count DESC
LIMIT 5;
"""
df_op2 = pd.read_sql(query_op2, conn)
display(df_op2)
```

	department	high_risk_patient_count	
0	ER	797	
1	Neurology	788	
2	ICU	783	
3	Orthopedics	770	
4	General	759	

Next steps: [Generate code with df_op2](#) [New interactive sheet](#)

Business Insight:

High-risk patients demand greater clinical focus and specialized equipment. The top departments on this list should be prioritized for senior staffing schedules and critical care resource allocation to mitigate clinical risks and prevent adverse outcomes.

```
# =====
# OPERATIONAL TASK 3 - Average LOS by Visit Type
# =====
query_op3 = """
-- Compare average duration across different visit settings (e.g., ICU vs OPD)
SELECT
    visit_type,
    ROUND(AVG(length_of_stay_hours), 2) AS avg_length_of_stay_hours
FROM visits
GROUP BY visit_type
ORDER BY avg_length_of_stay_hours DESC;
"""
```

```
df_op3 = pd.read_sql(query_op3, conn)
display(df_op3)
```

	visit_type	avg_length_of_stay_hours	
0	OPD	19.71	
1	ICU	19.53	
2	ER	19.41	

Next steps: [Generate code with df_op3](#) [New interactive sheet](#)

Business Insight:

Categorizing length of stay by visit type is essential for capacity forecasting. Prolonged stays in intensive care (ICU) or emergency (ER) settings cause systemic backups, delaying elective procedures and new admissions. This data informs shift planning and bed management strategies.

```
# =====
# OPERATIONAL TASK 4 - Frequent Flyers (> 3 Visits)
# =====
query_op4 = """
-- Flag patients who utilize the hospital network excessively
SELECT
    p.patient_id,
    p.age,
    p.chronic_flag,
    COUNT(v.visit_id) AS total_visits
FROM patients p
JOIN visits v ON p.patient_id = v.patient_id
GROUP BY p.patient_id, p.age, p.chronic_flag
HAVING COUNT(v.visit_id) > 3
ORDER BY total_visits DESC;
"""
df_op4 = pd.read_sql(query_op4, conn)
display(df_op4)
```

	patient_id	age	chronic_flag	total_visits	
0	3586	21	0	15	
1	3890	60	0	14	
2	4266	53	1	14	
3	4900	55	1	14	
4	146	59	0	13	
...	...	...	...	...	
3663	4984	42	0	4	
3664	4985	55	0	4	
3665	4987	41	1	4	
3666	4990	88	1	4	
3667	4995	33	0	4	

3668 rows × 4 columns

Next steps: [Generate code with df_op4](#) [New interactive sheet](#)

Business Insight:

Frequent readmissions place a heavy strain on hospital resources and frequently invite financial penalties from insurance networks. By identifying these "frequent flyers," particularly those with chronic conditions, care teams can transition them into proactive outpatient management programs, reducing ER strain.

```
# =====
# OPERATIONAL TASK 5 - Doctor Workload Analysis
# =====
query_op5 = """
-- Determine patient load distribution among physicians
SELECT
    doctor_id,
    COUNT(visit_id) AS total_patients_seen
FROM visits
```

```
GROUP BY doctor_id
ORDER BY total_patients_seen DESC
LIMIT 10;
"""
df_op5 = pd.read_sql(query_op5, conn)
display(df_op5)
```

	doctor_id	total_patients_seen	
0	180	290	
1	188	285	
2	113	283	
3	195	282	
4	111	280	
5	169	279	
6	161	273	
7	133	271	
8	152	269	
9	177	266	

Next steps: [Generate code with df_op5](#) [New interactive sheet](#)

Business Insight:

Physician burnout is a major liability in healthcare operations. Identifying doctors with disproportionately high patient loads allows clinical directors to rebalance shift scheduling and redistribute incoming cases, maintaining both staff well-being and high-quality patient care.

```
# =====
# FINANCIAL TASK 1 - Revenue Leakage by Insurance Provider
# =====
query_fin1 = """
-- Calculate the gap between what the hospital bills and what gets paid
SELECT
    p.insurance_provider,
    SUM(b.billed_amount) AS total_billed,
    SUM(b.approved_amount) AS total_approved,
    (SUM(b.billed_amount) - SUM(b.approved_amount)) AS total_revenue_leakage
FROM patients p
JOIN visits v ON p.patient_id = v.patient_id
JOIN billing b ON v.visit_id = b.visit_id
GROUP BY p.insurance_provider
ORDER BY total_revenue_leakage DESC;
"""
df_fin1 = pd.read_sql(query_fin1, conn)
display(df_fin1)
```

	insurance_provider	total_billed	total_approved	total_revenue_leakage	
0	MediCareX	1.345912e+08	1.001355e+08	34455694.29	
1	HealthPlus	1.301807e+08	9.625178e+07	33928965.67	
2	CareOne	1.307080e+08	9.699776e+07	33710234.47	
3	SecureLife	1.262890e+08	9.377089e+07	32518153.25	

Next steps: [Generate code with df_fin1](#) [New interactive sheet](#)

Business Insight:

This pinpoints the largest sources of revenue leakage. Insurance providers displaying the widest variance between billed and approved amounts severely impact the hospital's operating margin. The finance department must use this exact data variance to renegotiate payer contracts or audit disputed billing codes.

```
# =====
# FINANCIAL TASK 2 - Claim Rejection Rates by Department
# =====
query_fin2 = """
-- Measure the percentage of rejected claims localized to specific units
SELECT
    v.department,
```

```

COUNT(*) AS total_claims,
SUM(CASE WHEN b.claim_status = 'Rejected' THEN 1 ELSE 0 END) AS rejected_claims,
ROUND((SUM(CASE WHEN b.claim_status = 'Rejected' THEN 1.0 ELSE 0.0 END) / COUNT(*)) * 100, 2) AS rejection_percentage
FROM visits v
JOIN billing b ON v.visit_id = b.visit_id
GROUP BY v.department
ORDER BY rejection_percentage DESC;
"""
df_fin2 = pd.read_sql(query_fin2, conn)
display(df_fin2)

```

	department	total_claims	rejected_claims	rejection_percentage	
0	Orthopedics	4164	651	15.63	
1	Cardiology	4159	649	15.60	
2	General	4228	643	15.21	
3	Neurology	4165	627	15.05	
4	ER	4220	633	15.00	
5	ICU	4064	594	14.62	

Next steps: [Generate code with df_fin2](#) [New interactive sheet](#)

Business Insight:

High rejection rates isolated within specific departments usually indicate procedural errors rather than clinical ones—such as missing prior authorizations or inaccurate medical coding. Leadership should target departments with high rejection percentages for immediate Clinical Documentation Improvement (CDI) training.

```

# =====
# FINANCIAL TASK 3 - Payment Delays by Insurance Provider
# =====
query_fin3 = """
-- Identify which insurers take the longest to remit payment
SELECT
    p.insurance_provider,
    ROUND(AVG(b.payment_days), 1) AS avg_payment_days
FROM patients p
JOIN visits v ON p.patient_id = v.patient_id
JOIN billing b ON v.visit_id = b.visit_id
WHERE b.payment_days IS NOT NULL
GROUP BY p.insurance_provider
ORDER BY avg_payment_days DESC;
"""
df_fin3 = pd.read_sql(query_fin3, conn)
display(df_fin3)

```

	insurance_provider	avg_payment_days	
0	SecureLife	13.1	
1	HealthPlus	13.1	
2	MediCareX	13.0	
3	CareOne	13.0	

Next steps: [Generate code with df_fin3](#) [New interactive sheet](#)

Business Insight:

Operational liquidity relies on consistent cash flow. Insurers that consistently exhibit extended payment cycles disrupt the hospital's financial stability. The finance team should prioritize follow-up protocols and establish stricter payment timelines with these specific providers.

```

# =====
# FINANCIAL TASK 4 - Realized Cash Flow by Department
# =====
query_fin4 = """
-- Sum the actual cash brought in by each department
SELECT
    v.department,
    SUM(b.approved_amount) AS total_realized_revenue
FROM visits v
JOIN billing b ON v.visit_id = b.visit_id
WHERE b.claim_status = 'Paid'

```

```
GROUP BY v.department
ORDER BY total_realized_revenue DESC;
"""
df_fin4 = pd.read_sql(query_fin4, conn)
display(df_fin4)
```

	department	total_realized_revenue	
0	ER	51371575.27	
1	Orthopedics	51201835.72	
2	General	50784662.29	
3	Neurology	50394554.11	
4	Cardiology	49484551.33	
5	ICU	49029038.38	

Next steps: [Generate code with df_fin4](#) [New interactive sheet](#)

Business Insight:

Tracking fully realized revenue (as opposed to theoretical billed revenue) identifies the hospital's true profit centers. Understanding which departments generate actual cash empowers leadership to make data-backed decisions regarding capital investments and expansion planning.

```
# =====
# FINANCIAL TASK 5 - Dollar Value of Rejections
# =====
query_fin5 = """
-- Quantify the exact financial hit of rejected claims per department
SELECT
    v.department,
    COUNT(b.bill_id) AS rejected_count,
    SUM(b.billed_amount) AS lost_revenue
FROM visits v
JOIN billing b ON v.visit_id = b.visit_id
WHERE b.claim_status = 'Rejected'
GROUP BY v.department
ORDER BY lost_revenue DESC;
"""
df_fin5 = pd.read_sql(query_fin5, conn)
display(df_fin5)
```

	department	rejected_count	lost_revenue	
0	General	643	12797140.33	
1	Orthopedics	651	12773678.05	
2	Cardiology	649	12609305.91	
3	ER	633	12496172.68	
4	Neurology	627	12411191.52	
5	ICU	594	11799412.65	

Next steps: [Generate code with df_fin5](#) [New interactive sheet](#)

Business Insight:

While rejection percentages highlight process flaws, converting those rejections into hard dollar amounts reveals the true severity of the problem. Attaching a financial penalty to departmental inefficiencies creates the necessary urgency for immediate leadership intervention.

```
# =====
# DATA QUALITY TASK 1 - Invalid Age Checks
# =====
query_dq1 = """
-- Identify records where patient age violates logical bounds
SELECT
    COUNT(*) AS invalid_age_count
FROM patients
WHERE age < 0 OR age > 120 OR age IS NULL;
"""
df_dq1 = pd.read_sql(query_dq1, conn)
display(df_dq1)
```

invalid_age_count		
0	0	

Business Insight:

Age is a foundational clinical risk factor required for our upcoming machine learning models. Invalid or null data points will severely degrade predictive accuracy. Finding discrepancies here underscores the need for stricter front-desk data entry validation protocols.

```
# =====
# DATA QUALITY TASK 2 - Orphaned Visits (No Patient Match)
# =====
query_dq2 = """
-- Check relational integrity: Visits that don't map to a known patient
SELECT
    COUNT(*) AS orphaned_visits
FROM visits
WHERE patient_id NOT IN (SELECT patient_id FROM patients);
"""

df_dq2 = pd.read_sql(query_dq2, conn)
display(df_dq2)
```

orphaned_visits		
0	0	

Business Insight:

Relational data integrity is non-negotiable. An operational visit that cannot be traced back to a patient demographic record is unusable for longitudinal analysis. If present, this indicates a severe flaw in the hospital's Electronic Health Record (EHR) synchronization.

```
# =====
# DATA QUALITY TASK 3 - Illogical Billing Amounts
# =====
query_dq3 = """
-- Flag financial impossibilities (e.g., getting paid more than billed)
SELECT
    COUNT(*) AS logical_error_count
FROM billing
WHERE approved_amount > billed_amount;
"""

df_dq3 = pd.read_sql(query_dq3, conn)
display(df_dq3)
```

logical_error_count		
0	0	

Business Insight:

Insurance networks rarely approve payouts exceeding the initial hospital bill. Flagging records where this occurs points to systemic data extraction errors or faulty financial logging. These errors must be purged before training our claim outcome prediction models.

```
# =====
# DATA QUALITY TASK 4 - Invalid LOS Metrics
# =====
query_dq4 = """
-- Ensure length of stay is logically a positive duration
SELECT
    COUNT(*) AS invalid_los_count
FROM visits
WHERE length_of_stay_hours <= 0 OR length_of_stay_hours IS NULL;
"""

df_dq4 = pd.read_sql(query_dq4, conn)
display(df_dq4)
```

invalid_los_count		
0	0	

Business Insight:

An inpatient visit must inherently consume time. Because Length of Stay (LOS) is a primary predictor of hospital cost and operational efficiency, any zero or negative values represent corrupted data. These rows require immediate imputation or removal during our Phase 2 EDA process.

```
# =====  
# DATA QUALITY TASK 5 - Missing Demographics (Fairness Check)  
# =====  
query_dq5 = """  
-- Check for missing demographic variables crucial for fairness testing  
SELECT  
    COUNT(*) AS missing_demographics_count  
FROM patients  
WHERE gender IS NULL OR city IS NULL;  
"""  
df_dq5 = pd.read_sql(query_dq5, conn)  
display(df_dq5)
```

missing_demographics_count	
0	0

Business Insight:

Phase 4 of our project demands evaluating machine learning models for demographic fairness. If we lack substantial gender or regional data, we cannot guarantee our AI solutions treat all patient populations equitably, presenting both ethical and regulatory compliance risks.